

Problema de Rutas — BFS, UCS y A* (v2)

Documentación Técnica

Inteligencia Artificial

Índice

1. Descripción del Proyecto	2
2. Algoritmo A* (A-Star)	2
2.1. Fundamento	2
2.2. Heurística utilizada	2
2.3. Implementación A*	2
3. Comparativa de los Tres Algoritmos	3
4. Clase ResultadoBusqueda	3
5. Interfaz Gráfica	3

1. Descripción del Proyecto

Versión extendida del planificador de rutas que agrega el algoritmo **A*** con soporte de heurísticas. Implementa BFS, UCS y A* sobre el mismo grafo de ciudades mexicanas.

2. Algoritmo A* (A-Star)

2.1. Fundamento

A* combina el costo real acumulado $g(n)$ con una estimación heurística $h(n)$ de la distancia restante al destino:

$$f(n) = g(n) + h(n)$$

Para que A* sea óptimo, la heurística debe ser **admisibile**: nunca sobreestimar la distancia real.

2.2. Heurística utilizada

Se usa la distancia en línea recta (distancia euclidiana) entre ciudades, calculada con sus coordenadas geográficas aproximadas:

```
1 double heuristica(String ciudad, String destino) {
2     double[] coordCiudad = coordenadas.get(ciudad);
3     double[] coordDestino = coordenadas.get(destino);
4     double dLat = coordCiudad[0] - coordDestino[0];
5     double dLon = coordCiudad[1] - coordDestino[1];
6     // factor 111 km por grado de latitud
7     return Math.sqrt(dLat*dLat + dLon*dLon) * 111;
8 }
```

Esta heurística es admisible porque la línea recta es siempre menor o igual que la ruta real por carretera.

2.3. Implementación A*

```
1 Nodo resolverAEstrella(String origen, String destino) {
2     PriorityQueue<Nodo> abiertos =
3         new PriorityQueue<>(Comparator.comparingDouble(
4             n -> n.g + heuristica(n.ciudad, destino)));
5     Set<String> cerrados = new HashSet<>();
6
7     abiertos.add(new Nodo(origen, 0, null));
8
9     while (!abiertos.isEmpty()) {
10         Nodo actual = abiertos.poll();
11         if (cerrados.contains(actual.ciudad)) continue;
12         cerrados.add(actual.ciudad);
13
14         if (actual.ciudad.equals(destino)) return actual;
15     }
```

```

16         for (Arista a : grafo.get(actual.ciudad)) {
17             if (!cerrados.contains(a.destino)) {
18                 double nuevoG = actual.g + a.peso;
19                 abiertos.add(new Nodo(a.destino,
20                                     nuevoG, actual));
21             }
22         }
23     }
24     return null;
25 }

```

3. Comparativa de los Tres Algoritmos

Característica	BFS	UCS	A*
Estructura	Cola FIFO	Cola prioridad	Cola prioridad
Criterio	Saltos	$g(n)$	$f(n) = g(n) + h(n)$
Heurística	No	No	Sí
Óptimo en costo	No	Sí	Sí
Nodos explorados	Más	Medio	Menos
Velocidad	Media	Lenta	Rápida

4. Clase ResultadoBusqueda

Para unificar la salida de los tres algoritmos se usa una clase resultado:

```

1 class ResultadoBusqueda {
2     List<String> ruta;
3     double costoTotal;
4     int nodosExplorados;
5     long tiempoMs;
6 }

```

Esto permite mostrar comparativas de rendimiento entre los tres algoritmos para la misma consulta.

5. Interfaz Gráfica

- Tres botones: **BFS**, **UCS**, **A***.
- Muestra ruta, costo y nodos explorados para comparar eficiencia.
- La ruta se visualiza resaltada sobre el grafo de ciudades.